Understanding Liberty File Structure – Pin Directions, Logic Function, and Timing Arcs

INTRODUCTION:

This experiment demonstrates how to analyse a Liberty (.lib) file, which is the standard format used to describe standard-cell libraries in digital design. Each cell in the .lib file contains details about pins, their directions, Boolean logic functions, and timing arcs. By studying the library, students can learn how synthesis tools map RTL into physical cells and how timing is modelled for STA (Static Timing Analysis). In this experiment, we will explore the osu018_stdcells.lib file and identify details of some common cells (INV, NAND2, NOR2, and DFF).

OBJECTIVE:

After completing this lab, you will be able to:

- Understand the structure of a Liberty (`.lib`) file.
- Identify **pin directions** (input/output).
- Interpret **logic functions** defined in the `.lib`.
- Analyse **timing arcs** between input and output pins.
- Build a tabular observation of cell functionality

PROCEDURE TO CREATE A LAB:

Step 1: Open The Library

gvim osu018_stdcells.lib

```
coe-2@ip-10-100-19-29:~$ gvim osu018_stdcells.lib
```

Step 2: Search for a Cell

Use `/cell (` inside `gvim` to find cells.
Example:

```
/cell (INVX1)
```

Step 3: Identify Pin Directions

```
pin(A) {
    direction : input;
}
pin(Y) {
    direction : output;
    function : "!A";
}
```

This indicates A is input, Y is output, and the function is Y = NOT(A).

Step 4: Identify Logic Function

```
2959      function : "(!A)";
```

Check the function attribute under an output pin.

For example, in a NAND gate:

```
function : "!(A * B)";
```

This means Y = NOT(A AND B).

Step 5: Identify Timing Arcs

Inside the `timing()` block, check:

```
timing() {
    related_pin : "A";
     cell_rise(delay_template_3x3);
     cell_fall(delay_template_3x3);
}
```

This defines timing delay from pin **A** → **Y** for both rising and falling transitions.

```
2960      timing() {
2961          related_pin : "A";
2962          timing_sense : negative_unate;
2963          cell_fall(delay_template_5x5) {
```

295

Step 6: Repeat for Other Cells

Check **INVX1, NAND2X1, NOR2X1, DFFPOSX1** to cover both combinational and sequential examples.

OBSERVATION:

Cell Name	Pin	Direction	Logic Function	Timing Arc Example
INVX1	A	Input	Y = !A	A → Y (rise/fall)
INVX1	Y	Output		Output timing arcs
NAND2X1	A	Input	Y = !(A * B)	A → Y (rise/fall)
NAND2X1	B	Input		B → Y (rise/fall)
NAND2X1	Y	Output		Output timing arcs
NOR2X1	A	Input	Y = !(A + B)	A → Y (rise/fall)

Cell Name	Pin	Direction	Logic Function	Timing Arc Example
NOR2X1	B	Input		B → Y (rise/fall)
NOR2X1	Y	Output		Output timing arcs
DFFPOSX1	D	Input	Q = D @ clk↑	D → Q (setup/hold)
DFFPOSX1	CLK	Input	Sequential FF	CLK → Q (clk→q)
DFFPOSX1	Q	Output		Sequential timing

OUTPUT:

- Pin direction and logic function extracted from `osu018_stdcells.lib`.
- Timing arcs identified for combinational and sequential cells.
- Observation table summarizing the analysis.

CONCLUSION:

This experiment successfully demonstrated the analysis of a Liberty standard cell library. By examining pin directions, logic functions, and timing arcs, we observed how .lib files describe both combinational logic (INV, NAND, NOR) and sequential elements (DFF). The .lib serves as a critical input for synthesis, timing, and power analysis, ensuring accurate hardware implementation.